

module (transmitter) and the CAN bus receiver (tester).

CAN bus radar sensor module (transmitter). The Coin reads data from the HB100 Doppler radar and the onboard IMU, processes it, and transmits parameters, including relative speed, Doppler frequency, acceleration, orientation, and event detection, over the CAN bus. It also provides IoT capabilities via Wi-Fi, enabling real-time wireless monitoring through a user-friendly web

dashboard accessible from a laptop or smartphone.

The radar sensor module can be directly connected to the vehicle's OBD2 port or to the existing CAN network, enabling other ECUs to access radar and sensor data seamlessly.

For hardware interfacing, a TJA1050 CAN transceiver module connects the IndusBoard's TWAI pins (GPIO43 for TX and GPIO44 for RX) to the CAN bus. The HB100 output signal is amplified using an LM358 op-amp to generate clean pulses, which are then fed to the IndusBoard for frequency measurement and speed calculation. All processed data from the radar and IMU is transmitted over the CAN bus using twisted-pair wiring (CAN High and CAN Low) (see Fig. 5, lower section).

CAN bus receiver.

The CAN bus receiver is primarily intended for testing and debugging. It receives all

data transmitted by the radar module over the CAN bus and displays it clearly in the serial monitor. In actual vehicle deployment, this receiver unit can be omitted.

For testing, a receiver unit is constructed similarly, with the TJA1050 transceiver connected to the IndusBoard TWAI CAN bus pins. Both units are linked using twisted CAN High and CAN Low wires. The radar module continuously transmits data, while the receiver displays it on the serial monitor (see Fig. 5, top circuit).

OBD port connection. In a real vehicle, the radar module's CAN High and CAN Low lines can be directly connected to the OBD2 port (pin 6 to CAN High, pin 14 to CAN Low, pin 16 to +12V, and pins 4 and 5 to ground). This enables integration of radar, speed, IMU, and temperature data into the vehicle's existing CAN network for advanced driver-assistance systems (ADAS) applications. See Fig. 6 (top right) for the OBD interface board.

Software

For coding, two separate codes are developed. One is the CAN bus radar code that sends the data over the CAN bus and also updates it on the real-time Wi-Fi dashboard. The other

```

23 const uint32_t CAN_IMU_ID = 0x100;
24 const uint32_t CAN_RADAR_ID = 0x200;
25 const uint32_t CAN_EVENT_ID = 0x300;
26
27 // ----- HB100 DOPPLER RADAR -----
28 #define RADAR_PIN 4
29 volatile unsigned long pulseCount = 0;
30 unsigned long frequency = 0;
31 float radar_speed_mps = 0.0f;
32 float radar_speed_kmph = 0.0f;
33 const float HB100_CONSTANT_KMH = 19.49f;
34 unsigned long lastRadarMillis = 0;
35
36 void IRAM_ATTR radarISR() { pulseCount++; }
37
38 // ----- IMU -----
39 LSM303AGR_ACC_Sensor Acc(&Wire);
40 LSM303AGR_MAG_Sensor Mag(&Wire);
41
42 const float STATIC_THRESHOLD = 0.12f;
43 const float ACCELERATION_THRESHOLD = 0.25f;
44 const float BRAKE_THRESHOLD = -0.30f;
45 const float COLLISION_THRESHOLD = 2.5f;
46 const float FREEFALL_THRESHOLD = 0.25f;
47 const float ROLLOVER_ANGLE = 65.0f;
48
49 // ----- WiFi Configuration -----
50 const char* sta_ssid = "JioFiber-4G"; // - Your home WiFi SSID
51 const char* sta_password = "1234567890"; // - Your home WiFi Password
52
53 const char* ap_ssid = "CANBus-Radar"; // AP mode SSID
54 const char* ap_password = "12345678"; // AP mode Password (min 8 chars)
55
56 AsyncWebServer server(80);

```

Fig. 7: CAN bus ID settings and SSID/password configuration

```

4 // * 0x100 -> IMU (accel + temp)
5 // * 0x200 -> Doppler RADAR speed + frequency
6 // * 0x300 -> Events + Yaw/Roll/Pitch + Direction
7 // Real-time beautiful serial output
8 // =====
9
10 #include <Arduino.h>
11 #include <driver/twai.h>
12
13 const gpio_num_t TWAI_TX_PIN = GPIO_NUM_40;
14 const gpio_num_t TWAI_RX_PIN = GPIO_NUM_42;
15
16 const uint32_t CAN_IMU_ID = 0x100;
17 const uint32_t CAN_RADAR_ID = 0x200;
18 const uint32_t CAN_EVENT_ID = 0x300;
19
20 const char* event_names[] = {"STATIC", "ACCELERATION", "BRAKE", "COLLISION"};
21
22 String getDirection(float heading) {
23   heading = fmod(heading + 360.0, 360.0);
24   if (heading < 45 || heading >= 315) return "NORTH";
25   if (heading < 135) return "EAST";
26   if (heading < 225) return "SOUTH";
27   return "WEST";
28 }
29
30 void setup() {
31   Serial.begin(115200);
32   delay(200);
33   Serial.println("\n=== ESP32 CAN RECEIVER (Updated Structure) ===");
34   Serial.println("0x100 = IMU | 0x200 = Doppler Radar | 0x300 = Events + 0");
35
36   twai_general_config_t g_config = TWAI_GENERAL_CONFIG_DEFAULT(TWAI_TX_PIN,
37   twai_timing_config_t t_config = TWAI_TIMING_CONFIG_500KBITS();
38   twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();

```

Fig. 8: CAN bus receiver code snippet

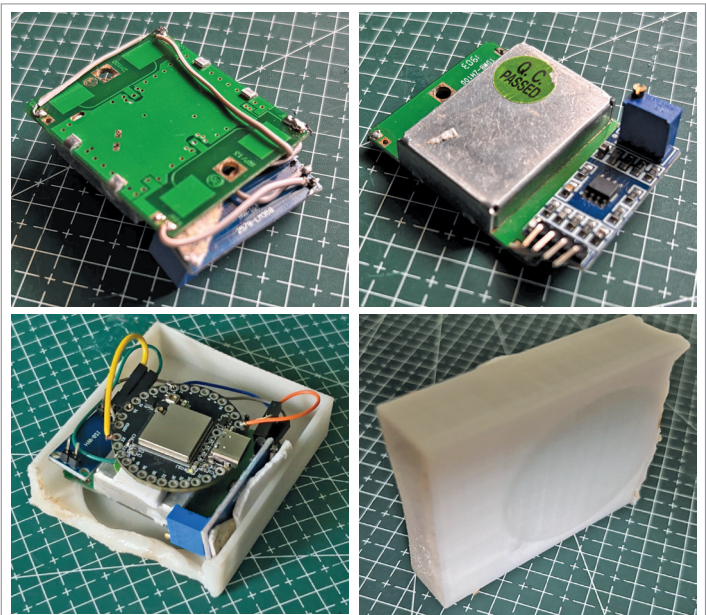


Fig. 9: Radar assembly and its fitting inside the enclosure